WALCHAND INSTITUTE OF TECHNOLOGY, SOLAPUR

DEPARTMENT OF INFORMATION TECHNOLOGY

MAJOR PROJECT – PHASE I

PROJECT DESIGN REPORT

ON

"Roopam Bag Mall Retail and Custom Bag Management System"

Submitted in partial fulfilment of the requirements for the award of Bachelor of Technology in Information Technology

Punyashlok Ahilyadevi Holkar Solapur University

Submitted by

Sr. No.	Student Name	Roll No.	E-mail	PRN No.
1	Atharv Tambat	B-59	atharvtambat20@gmail.com	2305111115
2	Sakshi Mohale	B-12	sakshimohale2005@gmail.com	2305111012
3	Shubham Jain	B-65	shubhamjain26tt@gmail.com	2405112007
4	Mokshit Gondaliya	B-67	gondaliyamokshit@gmail.com	2405112005

Under the Guidance of: Mr. S. R. Khole, Assistant Professor,Department of Information Technology.

Academic Year 2026–2027

CERTIFICATE

This is to certify that the Project Design Report entitled “Roopam Bag Mall Retail and Custom Bag Management System” has been prepared by **Atharv, Sakshi, Shubham, Mokshit** under the guidance of **Mr. S. R. Khole** as part of Major Project – Phase I, Department of Information Technology, Walchand Institute of Technology, Solapur, during academic year 2026–2027.

Guide Signature Project Coordinator Head of Department

ABSTRACT

Roopam Bag Mall Retail and Custom Bag Management System is a web application that connects the daily operations of a bag retailer with a structured customer customisation process. Separate stock records, sales records and informal design discussions can make it difficult to trace stock changes or preserve the exact configuration requested by a customer. The project addresses these workflow needs through a shared catalog, inventory ledger, purchases, counter sales, reports and customer design requests.

The architecture uses a React and TypeScript interface, a Spring Boot REST backend and a PostgreSQL database. Three.js and React Three Fiber render configurable Backpack, Laptop Bag and Duffel Bag models. The server validates model-specific options, derives request ownership from the authenticated identity, recalculates the estimated price and stores the design for later reconstruction. Administrators review requests through controlled status transitions. Retail transactions update stock and retain transaction and audit information.

A functioning local prototype is available. The integration record dated 5 October 2026 reports 317 passing backend tests with PostgreSQL regression suites enabled and a successful frontend build. These are recorded prototype results rather than evidence of production deployment. The remaining work includes formal user evaluation, production configuration, backup validation and presentation preparation. Custom bag submissions are enquiries for store review; they do not reserve inventory, collect payments or create retail orders.

ACKNOWLEDGEMENT

We express our gratitude to [Insert Guide Name] for guidance during the design and development of this project. We thank the Department of Information Technology, the project coordinator, the Head of Department and Walchand Institute of Technology, Solapur, for academic support and facilities. We acknowledge the authors of the research cited in this report and the maintainers of the open-source technologies used in the prototype. We also thank our families and peers for their encouragement.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	6
2	Literature Review and Existing System Analysis	7
3	Problem Definition and Requirement Analysis	11
4	Proposed System Architecture	14
5	Detailed System Design	17
6	Core Technical Design and Methodology	21
7	User Interface and Prototype Design	23
8	Technology Stack and Implementation Planning	25
9	Testing Strategy and Expected Outcomes	28
10	Conclusion and Future Scope	31
	References / Bibliography	31
	Appendices (if applicable)	32

List of Figures:

Figure 4.1 Overall system architecture — page 15

Figure 5.1 Standard custom request creation sequence — page 18

Figure 5.2 Principal database entity relationships — page 19

Figure 7.1 Key screen wireframes — page 24

List of Tables:

Table 2.1 Comparative literature review — page 10

Table 5.1 Module responsibilities and interfaces — page 17

Table 8.1 Technology stack — page 25

Table 8.2 Implementation status — page 26

Table 9.1 Requirements and acceptance tests — page 28

1.0 INTRODUCTION

1.1 Project Background

Bag retail involves maintaining product variants, monitoring stock, receiving purchases from suppliers, recording counter sales and answering customer enquiries. A customer may also want a bag with a particular material, colour, compartment layout, logo or text. These operations share business information but require different permissions and different levels of confirmation.

The project provides one browser application for customers and store staff. Customers browse the public storefront and submit saved custom bag designs. Cashiers operate the point of sale, inventory managers maintain stock and procurement, and administrators review custom requests. A local Java backend and relational database provide the common processing and storage layer.

1.2 Motivation and Need

The motivation is to make retail records and customer design communication consistent and traceable. A spreadsheet can represent quantities, but coordinating sales, purchase reversals, permissions and saved design options requires additional controls. A static product image also cannot show the effect of a customer's chosen colours and supported options. This project therefore combines transaction-based stock handling with interactive 3D configuration and a reviewable request record. Reduction in staff effort and improvement in customer understanding are evaluation goals; no field-study improvement percentage is claimed.

1.3 Project Overview

Roopam Bag Mall Retail and Custom Bag Management System combines a public storefront, customer registration, a 3D bag designer, customer request tracking, administrator review, catalog management, inventory history, supplier management, purchases, POS, sales history and sales reports. The customer interface and staff interface use the same secured API while presenting navigation appropriate to each role. Standard custom bag requests preserve the chosen configuration and store response separately from retail sales.

1.4 Aim and Objectives

Aim: To design and implement an integrated web prototype for traceable bag retail operations and customer custom bag requests.

O-01: Maintain products, categories, brands and suppliers with validated identifiers and active/inactive states.

O-02: Record purchases, sales and permitted reversals so that resulting stock quantities agree with their inventory movements in regression tests.

O-03: Enforce the CUSTOMER, CASHIER, INVENTORY_MANAGER and ADMIN permission matrix at the backend, including request ownership checks.

O-04: Allow customers to configure all three supported bag models, review an estimate, submit a standard request and reopen its saved 3D representation.

O-05: Permit only valid administrator review transitions and expose the saved status and store note to the owning customer.

O-06: Validate the prototype through automated backend tests, frontend build checks, workflow review and planned task-based usability testing.

1.5 Scope and Limitations

Included: the application architecture, relational schema, role matrix, transaction workflows, rule-based bag configuration, price estimation, private logo handling, screen design and prototype verification. The design supports one store and its local demonstration environment.

Outside the present scope: an online checkout, payment gateway, delivery tracking, custom request stock reservation, manufacturing automation, arbitrary free-form bag geometry, SPECIAL_DESIGN processing, AI recommendations and demand forecasting. Logo storage is local, internet connectivity is required for API operations, and the 3D interface depends on a WebGL-capable browser. Production load capacity and measured usability remain to be evaluated.

2.0 LITERATURE REVIEW AND EXISTING SYSTEM ANALYSIS

2.1 Review of Recent Research

The review covers seven papers published between 2021 and 2026 on retail inventory systems, customer management, replenishment and inventory optimisation. These studies range from

small-business web applications to large-scale analytics and attribute-level decision models. Their relevance to Roopam Bag Mall is assessed against the present prototype; reported benefits are not treated as measured project outcomes.

[1] Deng et al. (2023) describe Alibaba's integration of demand forecasting, inventory planning, price optimisation and product recommendations. Deep learning and simulation-based optimisation support coordinated retail decisions, with financial benefits reported for Alibaba's deployment. Its scale, data resources and operational infrastructure differ from a single-store prototype. For this project, the study supports keeping sales, purchase and stock information consistent as a foundation for later analytics; forecasting and recommendation algorithms remain outside the current scope.

[2] Muñoz Macas et al. (2021) review inventory management research for retail SMEs, covering systems, models, performance indicators and adoption challenges. The review identifies continued reliance on simple ERP tools, spreadsheets and manual records, illustrating the need for systems suited to limited resources. Its synthesis of earlier studies does not establish results for a bag retailer. The project therefore prioritises a practical shared catalog, transaction-linked stock history and accessible reports before advanced optimisation.

[3] Liew and Alduais (2022) develop a web-based inventory and customer management system for an online retail bakery using iterative development, PHP, AJAX, jQuery and MySQL. The application addresses manual ingredient records and customer service through automation of repetitive management tasks. Bakery ingredients and service workflows differ from bag variants and custom design enquiries. Its relevance is the integration of inventory and customer information in one web application, while the present project adds a separately controlled custom bag request workflow.

[4] Siagian and Suendri (2025) apply a Periodic Review System in a Next.js Progressive Web App for hardware-store order and inventory management. Demand and safety-stock information guide replenishment calculations, and the authors report improved stock tracking, delivery handling and offline access. The hardware-store setting and replenishment assumptions require local validation before transfer to bag retail. The study informs future scheduled stock review

and offline access; the current React prototype does not implement periodic replenishment or offline synchronisation.

[5] U. S. and P. Kaviya (2026) present a PHP/MySQL smart inventory system with administrator, manager and cashier roles. Its modules cover authentication, products, stock updates, purchase and sales tracking, alerts, analytics and reporting. The paper illustrates how modular web automation can connect routine retail operations. Its reported accuracy and error reductions require evaluation in the local store context. For Roopam Bag Mall, the relevant choices are role-controlled access and linked retail records; expiry alerts and the paper's performance figures are not claimed as prototype features or results.

[6] Koren and Peretz (2026) study colour-level fashion inventory using Markov decision processes. Their SRAMO optimisation procedure and SRIA diagnostic method use product recommendation transitions to assess and reduce imbalances across colour variants. The evaluation concerns fashion-platform data and a colour-balance objective, which do not establish an optimal assortment for a local bag shop. The work highlights the importance of retaining product-variant information for future analysis; Markov modelling and automated colour replenishment are outside the present implementation.

[7] Namazian, Betts and Stuckey (2025) combine demand prediction with stochastic optimisation for a beauty retailer operating under periodic inventory review. Forecast updates and re-optimisation account for uncertainty and lead times when determining replenishment quantities. The study reports better service and inventory outcomes in its case setting. Beauty-product demand and international supply constraints differ from this project's single-store context. The design implication is to retain reliable sales and purchasing histories for possible future forecasting, rather than claim an implemented predictive replenishment model.

2.2 Comparative Literature Review

Table 2.1 compares the reviewed approaches. Limitations describe applicability to the present project and do not imply that each paper intended to solve the same retail problem.

Table 2.1 Comparative literature review

Ref. / Year	Method / System	Key Findings	Limitations	Relevance
[1] / 2023	Integrated retail analytics	Coordinates forecasting and stock decisions	Large-scale Alibaba setting	Consistent retail data
[2] / 2021	Retail SME literature review	Maps systems, KPIs and adoption issues	Earlier studies; varied contexts	Practical stock records
[3] / 2022	Bakery web application	Links inventory and customer management	Bakery-specific workflow	Shared web application
[4] / 2025	Periodic review with PWA	Supports stock review and offline access	Hardware-store assumptions	Future replenishment review
[5] / 2026	Modular retail web system	Roles, stock, sales and reporting	Local results need validation	Role-controlled operations
[6] / 2026	Colour-based Markov models	Diagnoses variant inventory imbalance	Fashion-platform data	Retain variant information
[7] / 2025	Forecasting and optimisation	Adapts replenishment to demand uncertainty	Beauty-retail case	Future demand forecasting

2.3 Existing System Analysis

A manual register or spreadsheet workflow typically receives goods, records quantities, bills a sale and reconciles stock separately. Customer designs may be described through messages or sketches, which leaves the final specification dependent on interpretation. This is a baseline workflow model for analysis, not a claim that a formal on-site process audit has been completed.

The reviewed systems address inventory automation, customer management, replenishment and analytics in different retail settings [1]–[7]. A visual configurator instead focuses on selecting and displaying product variants. The project brings a bounded visual enquiry workflow into the same application as store operations, while keeping its data lifecycle distinct from sales. No feature-by-feature commercial product benchmark is claimed.

2.4 Research Gap and Design Implications

The reviewed work suggests four design implications. First, SME adoption challenges [2] favour a manageable web system built around everyday store tasks. Second, integrated customer management and retail automation [3], [5] support a common application with role-controlled access and transaction-linked records. Third, replenishment and optimisation studies [1], [4], [7] show why reliable sales and purchase histories should precede future forecasting or automated ordering. Fourth, colour-level inventory analysis [6] highlights the value of retaining variant information. These papers do not directly evaluate an interactive 3D custom bag request process, leaving a project-specific integration need: combine traceable retail operations with saved designs and administrator review. The contribution is application integration, without a claim of a new optimisation algorithm.

3.0 PROBLEM DEFINITION AND REQUIREMENT ANALYSIS

3.1 Problem Statement

A bag retailer needs consistent catalog, purchase, sales and stock records, while customers need a clear way to specify and revisit customised bag requirements. Disconnected records and informal descriptions make reconciliation and design review difficult. The project addresses these needs through a secured web system with transaction-linked stock updates, an interactive 3D designer and an owner-specific request review workflow.

3.2 Stakeholders and Users

Customer: registers, signs in, configures a standard bag, submits a request and reads only their own request details and store notes. Anonymous visitor: reads the public storefront and active brand/category advertising.

Cashier: reads product information and operates dashboard, POS and sales history/edit features. Inventory manager: additionally manages catalog, suppliers, inventory and purchases, accesses reports and cancels sales. Administrator: has staff capabilities and exclusive access to custom request review. Suppliers are business records rather than authenticated application users. The guide and evaluators review design evidence but do not require an application role.

3.3 Functional Requirements

FR-01: The system shall register a customer from validated name, email and password inputs, store a BCrypt hash and authenticate an active user to produce a JWT and role-aware session.

FR-02: The system shall return active brands and categories to public storefront visitors without exposing protected staff data.

FR-03: The system shall allow authorised managers/admins to create and update catalog and supplier records and reject invalid or duplicate identifiers.

FR-04: The system shall accept permitted stock movement inputs, validate quantities and product references, update stock and expose inventory history.

FR-05: The system shall save a purchase with supplier and line items, update stock, record supplier payments and reverse stock on valid cancellation.

FR-06: The system shall create and edit a counter sale, calculate item totals and discounts, record payment and stock changes, and retain audit information. Only manager/admin roles may cancel a sale.

FR-07: The system shall return date-based and grouped sales reports for authorised roles, keeping completed and cancelled semantics explicit.

FR-08: The system shall display Backpack, Laptop Bag and Duffel Bag configurations with supported options and a price estimate, then require review before submission.

FR-09: The system shall validate a multipart design request, derive its customer from the JWT, recompute pricing and return a saved request number. It shall accept only supported PNG/JPEG logos up to 2 MB.

FR-10: The system shall expose request history, reconstructed configuration and protected logos only to the owning customer or authorised administrator.

FR-11: The system shall accept only supported administrator review transitions, save the store note and return the updated status to the customer on retrieval.

FR-12: The system shall return structured validation, authentication, authorisation and not-found responses without exposing internal exception details.

3.4 Non-Functional Requirements

NFR-01 Security: every protected endpoint shall enforce its role rule; all negative role and cross-owner scenarios in the regression matrix shall pass. Invalid/expired tokens shall be rejected and raw passwords shall never be returned.

NFR-02 Integrity and reliability: invalid retail operations shall roll back their database changes; verified purchase/sale reversal cases shall restore the expected stock. A failed request save shall trigger cleanup of its newly stored logo.

NFR-03 Performance target: on a documented local test machine with 10 concurrent users and 1,000 products, the 95th-percentile response time for catalog reads should be at most 2 seconds. This is a proposed acceptance target; no load-test result is available.

NFR-04 Usability target: at least 4 of 5 representative participants should complete a standard design submission and a staff sale task without assistance. Review the interface at 360, 768 and 1280 CSS-pixel widths, allowing horizontal scrolling inside staff data tables.

NFR-05 Privacy and operations: protected logos and customer notes shall remain inaccessible to another customer. Production readiness requires HTTPS, secret configuration and a documented database-plus-upload backup and restore trial; these deployment checks are planned.

NFR-06 Maintainability: schema changes shall use versioned Flyway migrations; the backend regression suite and frontend type/build checks shall pass before release. Larger catalog and request-list loads shall be measured before any scale claim.

3.5 Use Cases and Constraints

UC-01 Configure a bag. Actor: CUSTOMER. Preconditions: active login and WebGL support. Main flow: choose model and options, inspect preview and estimate, review, submit, then reopen the saved request. Alternate flows: reject unsupported options or invalid logo; after a network failure, inspect request history before retrying to avoid duplicate submissions.

UC-02 Record a sale. Actor: CASHIER, INVENTORY_MANAGER or ADMIN. Preconditions: authorised session and valid product stock. Main flow: select products, enter quantities and discount, choose payment method, submit and inspect the saved sale. Invalid quantities or insufficient stock return an error.

UC-03 Review a request. Actor: ADMIN. Main flow: open inbox, filter requests, inspect saved configuration, choose an allowed next state and save a store note. A terminal-state reversal is rejected.

UC-04 Receive and cancel a purchase. Actor: INVENTORY_MANAGER or ADMIN. Main flow: choose supplier and items, save receipt and record payment; cancel only when reversal is valid. Dependencies: PostgreSQL availability, private upload storage and matching API deployment configuration. Formal approval of the design remains an academic review activity.

4.0 PROPOSED SYSTEM ARCHITECTURE

4.1 Overall System Architecture

Figure 4.1 shows the project architecture. Browser interfaces exchange JSON or multipart requests with the Spring Boot API. Three-dimensional rendering runs in the browser. Business services persist relational data through JPA and store logo files through a private file service. No payment gateway, manufacturing service or external AI API participates in this flow.

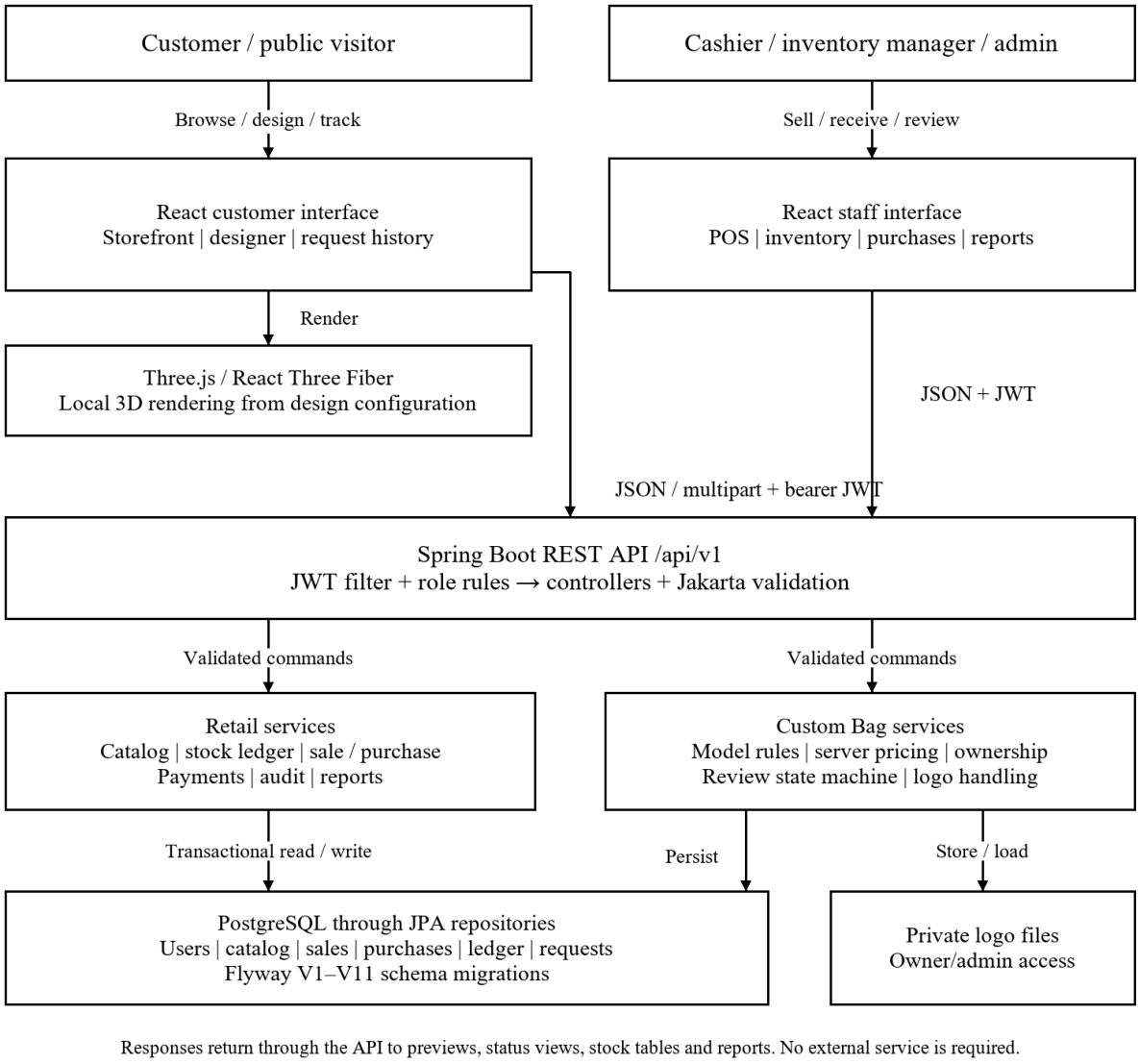


Figure 4.1 Overall system architecture

4.2 Architecture Components

Presentation layer: React Router separates public, customer and staff routes; protected and role routes control navigation. TanStack Query manages server data and Axios handles API communication. Customer and staff layouts present the functions appropriate to the signed-in user.

API and security layer: the JWT filter validates authentication, Spring Security applies endpoint permissions and controllers accept typed DTOs with Jakarta validation. CurrentUserService supplies trusted actor identity. Error handlers map failures to safe client responses.

Business layer: catalog services maintain master data; purchase and sale services coordinate amounts, payments, stock and reversals; report services aggregate sales; custom request services validate options, calculate estimates, enforce ownership and administer review states.

Persistence layer: PostgreSQL stores relational records and constraints, Flyway maintains schema versions, and Hibernate validates mapping compatibility. A separate local logo directory contains image data referenced by request records. Database and uploads together form the recoverable application state.

4.3 End-to-End Data and Control Flow

Customer data flow: (1) collect enumerated design options and optional branding; (2) render the current configuration locally; (3) submit a JSON request part and optional logo part; (4) validate role, fields and bag rules; (5) derive the customer identity and recompute the estimate; (6) store the request and logo reference; (7) return request details; (8) reconstruct the saved design on later retrieval.

Retail data flow: a purchase or sale DTO enters an authorised controller, service validation resolves product/supplier data, a transactional service updates the business document and stock ledger, and the response updates the browser view. Reports read persisted records with explicit date and status filters.

Control flow differs from payload flow: JWT and role checks decide whether processing may proceed; validation selects success or error paths; the review state machine selects permitted transitions; transaction commit/rollback determines persistence. A status change in a custom request never triggers a stock or payment operation.

4.4 Design Rationale

A layered modular backend fits a single-store prototype and keeps transaction boundaries within one application. A relational database matches the many linked records and constraints in retail operations. Client-side parametric rendering gives immediate visual feedback without a rendering server. Server validation is retained because a browser can send modified requests. These choices respond to bounded configuration [2], [4], integration concerns [8], [9] and prototype resource limits. A microservice deployment would add operational work without a demonstrated need at this scale.

5.0 DETAILED SYSTEM DESIGN

5.1 Module Identification and Responsibilities

Table 5.1 identifies module boundaries and the main data exchanged. Controllers and security form shared dependencies. Validation failures stop processing; missing records return not-found responses; unexpected errors are logged on the server and represented safely to the client.

Table 5.1 Module responsibilities and interfaces

Module	Purpose	Inputs	Processing	Outputs	Dependencies
Auth	Identity	Login / JWT	Hash and verify	User / token	Users / security
Catalog	Master data	Product / SKU	Validate / save	Catalog records	Category / brand
Stock	Trace stock	Movement DTO	Validate / adjust	Ledger / balance	Products
Purchases	Receive goods	Supplier / items	Totals / receipt / reversal	Purchase / payments	Stock / suppliers
Sales	Counter billing	Cart / discount	Totals / stock / audit	Sale / payment	Stock / actor
Reports	Sales analysis	Dates / grouping	Filter / aggregate	Report DTOs	Sales / items
Designer	Visual choices	Bag options	Render / review	3D / request DTO	Three.js / auth
Requests	Save / review	DTO / logo / state	Validate / price / transition	Saved design / note	Users / files / DB

5.2 Module Interaction Design

Figure 5.1 details the standard customer request interaction. Validation precedes persistence; the service obtains ownership from the signed-in identity rather than a customer identifier supplied

by the browser. Logo cleanup compensates for a failed database save. Administrator review is a later authenticated request against the saved record.

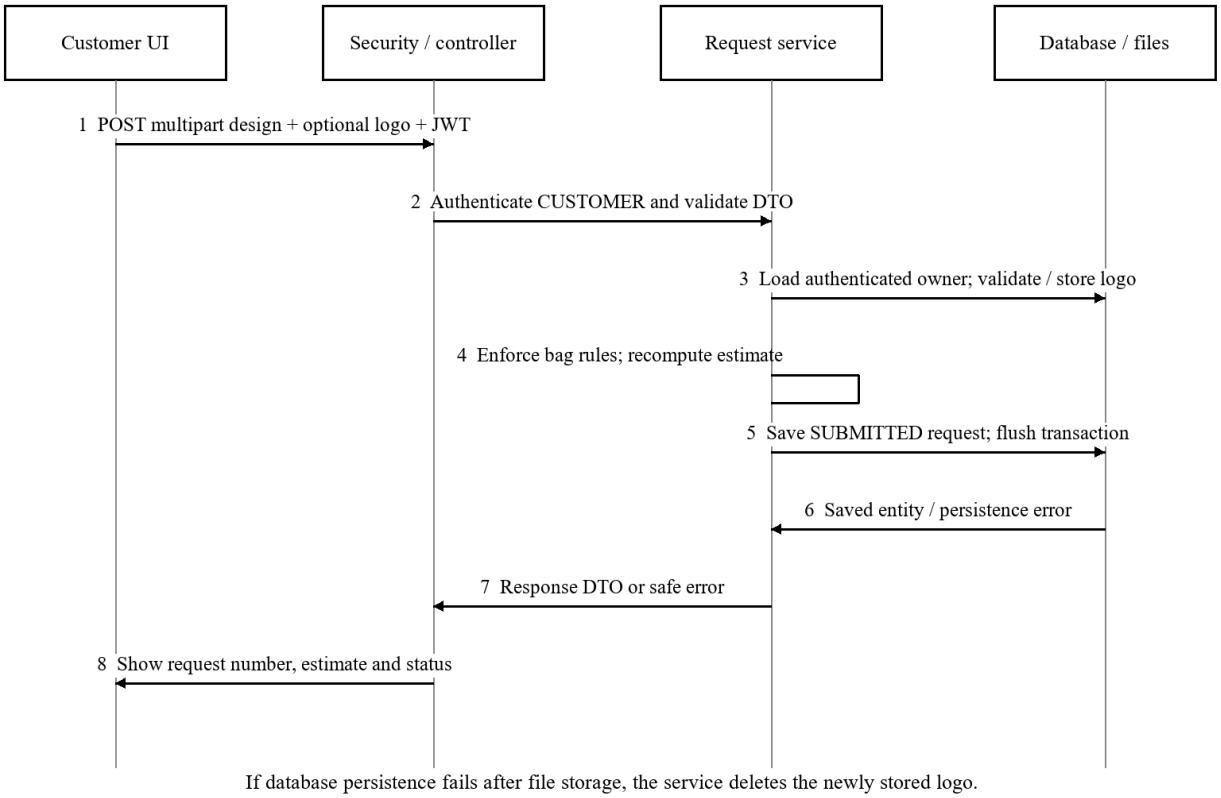


Figure 5.1 Standard custom request creation sequence

5.3 Data / Database / Workflow Design

Figure 5.2 shows the principal relational design. A category has many products, a brand can identify many products, and each sale/purchase contains multiple item rows linked to products. Supplier contacts and payments extend supplier and purchase records. Stock movements are represented by inventory transactions; sale audit history records relevant changes and actors.

A user owns many custom_bag_requests through customer_user_id. Each request stores enumerated model, size, material and colours, supported booleans, compartment count, optional logo/text metadata, estimated price, review status, notes and timestamps. request_number is unique. Logo bytes remain in protected file storage; the database keeps a generated reference.

Database constraints include primary/foreign keys, unique user email and request numbers, and checks on supported enum values and nonnegative estimates. Product SKU uniqueness and service validation protect catalog identity. Model-specific constraints are also checked in the service because permitted compartments and features vary by bag type.

The review workflow is SUBMITTED → REVIEWING → APPROVED → COMPLETED; SUBMITTED and REVIEWING may also become REJECTED. COMPLETED and REJECTED are terminal. CANCELLED exists in the schema but the current administrator transition map exposes no path into it. A stored request is not a sales order.

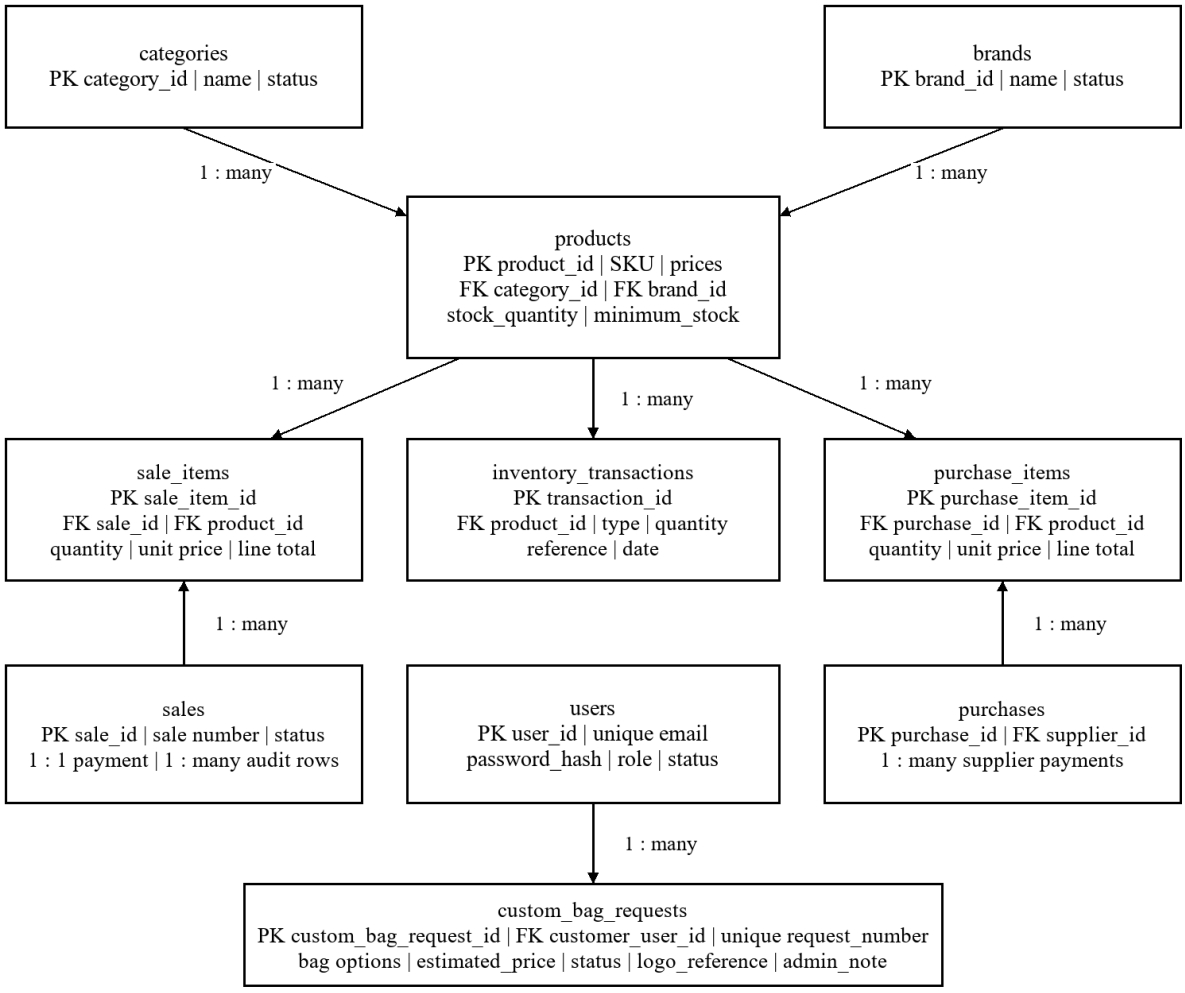


Figure 5.2 Principal database entity relationships

5.4 API / Interface Design

All application endpoints use /api/v1. JSON is the default representation; protected calls send Authorization: Bearer <token>. POST /auth/register and POST /auth/login are public; GET /auth/me returns authenticated user details. GET /public/storefront/brands and /categories provide public advertising records.

Catalog and retail interfaces: /products, /categories, /brands and /suppliers expose list/create and identifier-based detail/update/status operations. /inventory-transactions supports history and validated movement creation. /purchases supports receipt, edit, cancellation and nested payment operations. /sales supports creation, edit, detail, dashboard and cancellation. /sales/reports exposes daily, date-range, by-product, by-category, by-payment-method and cancelled views.

POST /custom-bag-requests accepts multipart/form-data with an application/json part named request and an optional image part named logo. The request specifies requestType, bagType, size, material, bodyColor, pocketColor, strapColor, functional options, compartmentCount, branding and customerNotes. The server response includes saved identifiers, configuration, estimatedPrice and status.

GET /custom-bag-requests/mine and /mine/{id} return owner-specific history and details; /mine/{id}/logo streams a protected logo. Administrator list/detail/logo paths use /custom-bag-requests/admin; PATCH /admin/{id} accepts status and adminNote. Cross-owner lookups return 404. Authentication failure returns 401, forbidden roles return 403, and validation failures return a structured client error. Hardware communication protocols are not applicable because this is a browser-based software system.

5.5 Security and Error Handling

Access control is enforced twice: role-aware navigation helps users find permitted functions, and backend request rules independently enforce permissions. BCrypt protects stored passwords. JWT validation and current-user lookup establish identity; request ownership filters prevent a customer from reading another customer's request or logo. The server, not the submitted payload, determines the review actor and request owner.

Jakarta constraints and service checks validate required fields, monetary and quantity rules, model-specific combinations and review transitions. Upload handling limits image type and size

and uses generated file references. Unexpected exceptions are logged internally; client messages omit stack traces and internal exception text.

Transactional retail processing provides rollback on failures, while new-logo cleanup addresses file/database mismatch during request creation. Deployment hardening requires HTTPS, protected secrets, restricted filesystem permissions, backup/restore checks and a documented log-retention policy. An explicit concurrency/load review remains necessary before deployment; a passing functional regression does not establish production race safety.

6.0 CORE TECHNICAL DESIGN AND METHODOLOGY

6.1 Technical Approach

The core method is deterministic, parameter-driven bag configuration combined with transactional retail logic. A finite configuration object controls browser geometry and materials. The backend checks that the object belongs to a supported solution space and calculates an authoritative estimate from fixed pricing rules. There is no training dataset, learned model or generative geometry pipeline.

Saved configuration values drive the same model components when a customer or administrator reopens a request. Review states constrain what the store may do next. Retail sale/purchase logic separately updates stock and monetary records so that a design enquiry cannot accidentally change store availability.

6.2 Detailed Processing Steps

Algorithm 6.1 Standard custom request creation

Input: authenticated CUSTOMER, design DTO and optional logo. Output: saved request DTO or validation/error response.

1. Validate required enum fields, text lengths and supported logo constraints.

2. Check model rules: Backpack/Duffel allow 1–4 compartments; Laptop Bag allows 1–3. Reject side pockets for Laptop Bag and laptop padding for Duffel Bag. Reject SPECIAL_DESIGN.

3. Require logo position when a logo exists; require text colour and position when trimmed custom text is nonempty. Resolve the customer from the authenticated identity.

4. Store the valid logo under a generated reference. Calculate the estimate from the server rule table; do not accept a client-provided price as authoritative.

5. Generate the request number; populate configuration, owner, timestamps and SUBMITTED status. Save and flush within the service transaction.

6. If persistence raises an error, delete the newly stored logo and propagate the error. Otherwise return the saved DTO. On later reads, filter by owner or require ADMIN.

Price formulation: $E = B + S + M + 100(c - 1) + 100f + 150s + 200l + 150w + 200g + 100t$, in INR. Here c is compartment count and f, s, l, w, g, t indicate front pocket, side pockets, laptop padding, water resistance, logo and nonempty text respectively.

Base B: Backpack 1200, Laptop Bag 1400, Duffel Bag 1600. Size S: Small 0, Medium 150, Large 300. Material M: Polyester 0, Canvas 150, Leather 500. The result is represented to two decimal places. Example: a large Canvas Backpack with two compartments, side pockets, water resistance, logo and text has $E = 1200 + 300 + 150 + 100 + 150 + 150 + 200 + 100 =$ INR 2350.00.

Algorithm 6.2 Retail stock processing: validate the authorised sale/purchase and line quantities; resolve active products and business rules; calculate line totals and permitted discounts; verify stock availability for outward movements; persist the document, associated payment/audit data and stock movement within a transaction. A permitted cancellation records the inverse stock effect and preserves the document's cancelled state for reporting. Duplicate cancellation must be rejected.

6.3 Project-Specific Design Details

Backend design: Spring controllers use typed DTOs, services express business rules and repositories persist JPA entities. Sale totals use decimal arithmetic; the current sale calculation subtracts the chosen discount from subtotal and does not add a separate tax amount. Product/category report amounts are gross item amounts, while completed-sale grand totals reflect sale-level discounts.

Frontend design: TypeScript models connect forms to the API. React Query refreshes server data, and session cleanup clears cached identity data after a 401. Three model components share rendering utilities but retain model-specific geometry. Saved colours, material, size, branding positions and options reconstruct the request.

Storage and recovery: Flyway V1–V11 define the prototype schema; Hibernate validates the mappings. Protected logo files and relational records must be migrated and backed up together. An annual sequence generates readable request numbers. No new dataset acquisition or AI training is required for this methodology.

6.4 Technical Contribution

The engineering contribution is the integration of a saved, reconstructable bag configuration and constrained administrator review into a retail management prototype with distinct role permissions and stock workflows. Compared with the different emphases of the reviewed studies, it combines a bounded customer interface, authoritative backend rules and an operational handover record in one application. The contribution is demonstrated through implemented modules and regression scenarios; improved user satisfaction, business revenue and manufacturing efficiency remain hypotheses for later evaluation.

7.0 USER INTERFACE AND PROTOTYPE DESIGN

7.1 User Journey and Navigation

Customer journey: /home → register or login → /custom-bag → review configuration → submit → /my-custom-bags → saved detail and store response. The designer allows supported model selection, preview rotation/zoom, option changes and optional logo/text. Submission requires a customer login.

Staff journey: login → role-appropriate /dashboard → catalog, inventory, purchases, POS, sales or reports. An administrator additionally opens /custom-bag-requests to review the same saved customer design. Cashiers do not receive report or cancellation controls; inventory managers do not receive the administrator request inbox. Forbidden access leads to an appropriate return route.

7.2 Screen Designs / Wireframes

Figure 7.1 presents original schematic wireframes for the designer, POS and administrator review screens. They document the layout and actions of the implemented route families rather than claim to be screenshots. The designer groups configuration, 3D feedback and submission review; POS separates product selection from the cart; review presents stored design information alongside permitted status controls.

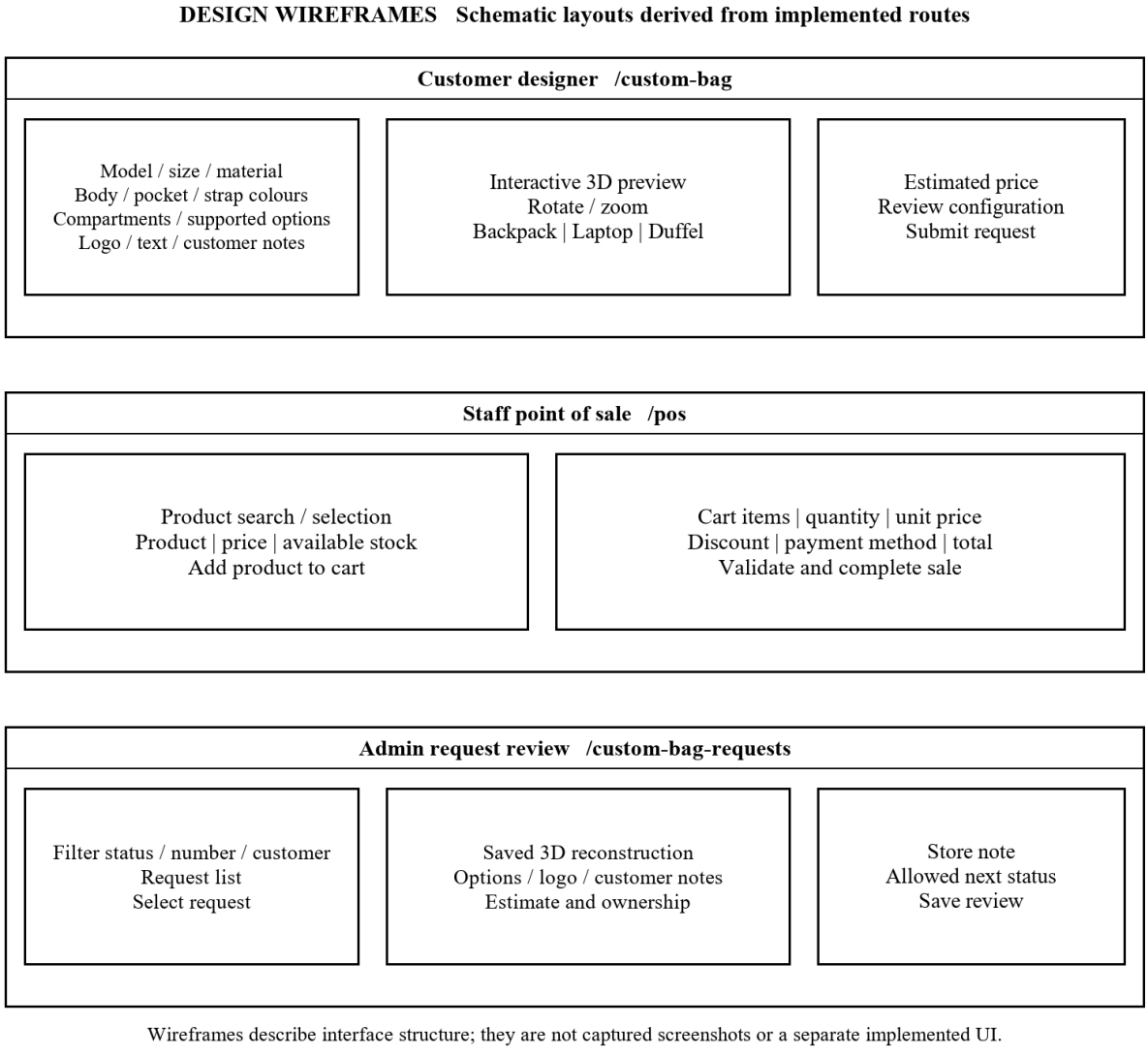


Figure 7.1 Key screen wireframes

7.3 Prototype Status

Functioning prototype: public storefront, role-aware authentication, three 3D bag models, standard request submission with optional logo, customer history/detail, administrator review, catalog, inventory, suppliers, purchases, POS, sales and reports are represented by implemented source modules.

Recorded integration review on 5 October 2026 exercised the customer and administrator saved preview workflow through APPROVED. Completion and terminal transition rejection were verified in transactional PostgreSQL tests; the persistent presentation example was not manually advanced to COMPLETED. The wireframes in this report are design representations. Production deployment, formal usability scoring and load benchmarks are planned.

8.0 TECHNOLOGY STACK AND IMPLEMENTATION PLANNING

8.1 Proposed Technology Stack

Table 8.1 records technologies declared in the repository. Frontend versions are the dependency specifications, not a claim that every compatible patch has been evaluated. Selection emphasises typed interfaces, relational integrity and browser-based 3D interaction.

Table 8.1 Technology stack

Category	Proposed Technology	Version (if known)	Purpose / Justification
Programming Language	Java; TypeScript	21; ~6.0.2	Backend rules and typed browser code
Framework / Library	Spring Boot; React; Vite	3.3.3; ^19.2.8; ^8.2.2	REST services and component interface
Database / Storage	PostgreSQL; Flyway; local files	PostgreSQL 18.6 in recorded demo; V1–V11	Relational records, migrations and logos
Tools / Platform	Maven; npm; Git; Tailwind CSS	Maven 3.9+ setup; Tailwind ^4.3.3	Builds, version control and styling

Category	Proposed Technology	Version (if known)	Purpose / Justification
Hardware / External API	WebGL browser; Three.js / Fiber / Drei	^0.186.1 / ^9.8.1 / ^10.7.9	3D preview; no external API required

8.2 Implementation Status

Table 8.2 separates implemented prototype modules from remaining release activities. Evidence is located in the repository and its integration record dated 5 October 2026. No fresh application test execution is implied by preparing this report.

Table 8.2 Implementation status

Activity / Module	Current Status	Evidence / Deliverable	Next Action
Auth / role matrix	Completed prototype	SecurityConfig; auth and role tests	Review deployment secrets
Catalog / suppliers / stock	Completed prototype	Services, migrations and screens	User acceptance review
Purchases / sales / reports	Completed prototype	Service and PostgreSQL tests	Concurrency/load checks
3D designer / saved requests	Completed prototype	Three model files; request tests	Usability evaluation
Admin review states	Completed prototype	Transition map and tests	Confirm presentation sample
Documentation	In progress	README and design report	Guide review and metadata
Staging / recovery / load	Not started in supplied evidence	Planned Section 8.3 checks	Configure and measure
Formal user evaluation	Not started in supplied evidence	Planned five-participant tasks	Recruit and record results

8.3 Milestones and Work Plan

The following four-week plan starts after the guide's review; week numbers are relative because an approved submission calendar has not been supplied.

Week 1: review requirements and report with the guide, confirm genuine store contact details, complete team metadata and obtain representative user tasks. Deliverable: reviewed design and agreed acceptance checklist.

Week 2: conduct task-based usability and accessibility review, record task success/time, test responsive layouts and address confirmed defects. Deliverable: participant observations and prioritised fixes.

Week 3: configure a staging deployment with HTTPS and environment secrets; run load/concurrency checks and a database-plus-upload restore trial. Deliverable: measured performance and recovery evidence.

Week 4: rerun regression/build checks after fixes, collect approved demonstration screenshots, finalise report pagination and prepare the viva demonstration. Deliverable: reviewed release candidate, documentation and presentation. Deployment is conditional on successful acceptance checks.

8.4 Feasibility and Risks

Technical feasibility: the local prototype uses widely supported web and relational technologies, and the existing verification record supports functional feasibility. WebGL support, large frontend chunks and unmeasured concurrent writes remain practical constraints.

Economic feasibility: the chosen frameworks are open source and do not require a paid external API. Hosting, backup storage, maintenance and user-support costs still need a deployment budget; no cost-saving amount is claimed. Operational feasibility depends on staff entering accurate source records and reviewing custom requests before committing to manufacture.

Key risks and mitigations: stock races under concurrent requests require targeted concurrent transaction testing and locking review; loss of logo files requires coordinated database/file backups; oversized 3D bundles require load measurement and later optimisation; unverified store

details require owner confirmation; unsupported option combinations require server validation; schedule risk is controlled by freezing the current scope before formal evaluation.

9.0 TESTING STRATEGY AND EXPECTED OUTCOMES

9.1 Testing Plan

Unit tests cover pricing rules, validation, number generation, service operations and entity behaviour. Controller/security tests verify request mapping, safe errors and the permission matrix. PostgreSQL regressions run real migrations and transactional fixtures for authentication, sales, purchases and custom request persistence. Test fixtures must use an isolated development/test database, never a production database.

System testing follows customer design submission and staff retail journeys, including invalid inputs and forbidden roles. Planned usability evaluation measures task completion and assistance for at least five representative participants. Planned performance/recovery checks measure latency under stated load and restoration of both records and logos. Table 9.1 defines concrete acceptance scenarios.

Table 9.1 Requirements and acceptance tests

Test ID	Requirement / Module	Test Input / Scenario	Expected Result	Metric / Acceptance Criterion
T-01	FR-01 Auth	Valid login; invalid/expired token	Login succeeds; bad tokens fail	Correct identity; 401 for invalid token
T-02	FR-01 / 10 Roles	Cross-role API; another owner ID	403 role denial; owner 404	All negative role cases pass
T-03	FR-03 Catalog	Duplicate SKU; invalid price	Reject invalid catalog request	No duplicate/invalid record

Test ID	Requirement / Module	Test Input / Scenario	Expected Result	Metric / Acceptance Criterion
T-04	FR-04 / 05 Stock	Purchase two units; cancel receipt	Stock increases then reverses	Expected ledger; original quantity
T-05	FR-06 Sales	Create/edit sale; manager cancel	Adjust stock; inverse on cancel	Correct stock and audit actor
T-06	FR-06 Security	Cashier requests sale cancellation	Forbidden; no mutation	403; stock unchanged
T-07	FR-08 / 09 Pricing	Valid model; altered client estimate	Server calculates its price	Exact two-decimal rule total
T-08	FR-09 Validation	Laptop side pockets; invalid logo	Reject unsupported input	No saved invalid request/file
T-09	FR-10 Saved design	Submit then retrieve own request	Equivalent options and preview	All persisted values match
T-10	FR-11 States	Allowed states; reopen terminal	Accept allowed; reject terminal	Correct state and saved note
T-11	FR-07 Reports	Completed/cancelled sales by date	Correct filtered aggregates	Match fixtures and gross/net labels
T-12	FR-12 Errors	Validation error; unexpected failure	Safe structured response	No stack trace/internal message
T-13	NFR-03 Load	10 users; 1,000 products	Catalog p95 <= 2 seconds	Planned measured load trial

Test ID	Requirement / Module	Test Input / Scenario	Expected Result	Metric / Acceptance Criterion
T-14	NFR-04 Usability	5 users complete key tasks	At least 4 succeed unaided	Planned observed task success
T-15	NFR-05 Restore	Restore database plus upload files	Saved requests/logos accessible	Planned record/file checks

9.2 Evaluation Criteria

Functional acceptance: all applicable automated cases pass without failures/errors; sale and purchase scenarios produce the expected ledger changes; owner/role restrictions reject every negative security case; saved requests reconstruct equivalent design values; invalid review transitions fail.

Recorded baseline: the 5 October 2026 integration record reports 317 tests passed with all opt-in PostgreSQL suites enabled. The default run reported 305 cases, of which 296 passed and 9 were skipped. The frontend lint run had zero errors and 15 warnings; the build passed with a large-chunk advisory. These dated results establish a baseline, not a current production certificate.

Release evaluation still requires the NFR-03 response-time target, NFR-04 task-success target and a demonstrated backup restore. No measured performance, SUS score or participant success rate is available in the current evidence.

9.3 Expected Deliverables

Achieved prototype outputs: Spring Boot API, React/TypeScript interface, PostgreSQL migrations, three configurable 3D bag models, saved standard custom request workflow, staff retail modules and automated regression sources. Available design outputs include the architecture, interaction, schema and interface representations in this report.

Planned release outputs: reviewed user-evaluation results, load/concurrency evidence, staging deployment configuration, restore demonstration, approved screenshots, final presentation and

completed personal/certificate metadata. A deployment URL and signed academic approval are not claimed.

10.0 CONCLUSION AND FUTURE SCOPE

10.1 Design Summary

The design connects customer customisation and store operations through a shared secured application. Parametric 3D models provide immediate visual feedback, server rules establish valid configurations and estimates, and saved owner-specific requests support review and later reconstruction. Transactional retail services maintain sale/purchase effects and stock history. The architecture and distinct workflows address traceability without treating an enquiry as a paid order.

10.2 Readiness for Implementation

The repository contains an implemented local prototype and a dated successful integration baseline. Academic approval of the final title and design remains with the guide and department. Immediate next steps are to insert team/guide details, review store information, complete user and performance evaluation, validate recovery and prepare the final demonstration. The existing schema, roles and module boundaries provide a basis for these activities; production readiness is not inferred from prototype completeness.

10.3 Future Scope

Future work may add offline staff operation with conflict resolution, barcode scanning, demand forecasting after sufficient quality data is available, multi-store stock transfers, automated customer notifications, accessibility improvements, configurable pricing administration and richer material previews. A separately designed order/quotation module could connect approved requests to payments and fulfilment. Such work requires explicit consent, security review, new business rules and testing rather than changing the meaning of existing request records.

REFERENCES / BIBLIOGRAPHY

[1] Y. Deng, X. Zhang, T. Wang, L. Wang, Y. Zhang, X. Wang, S. Zhao, Y. Qi, G. Yang, and X. Peng, “Alibaba Realizes Millions in Cost Savings Through Integrated Demand Forecasting,

Inventory Management, Price Optimization, and Product Recommendations," INFORMS Journal on Applied Analytics, vol. 53, no. 1, pp. 32–46, 2023, doi: 10.1287/inte.2022.1145.

[2] C. V. Muñoz Macas, J. A. Espinoza Aguirre, R. Arcentales-Carrion, and M. Peña, "Inventory Management for Retail Companies: A Literature Review and Current Trends," Proceedings of the 2021 2nd International Conference on Information Systems and Software Technologies (ICI2ST), IEEE, pp. 71–78, 2021.

[3] C. C. Liew and N. Alduais, "Development Web-Based Inventory and Customer Management System for Retail Bakery Online," Applied Information Technology and Computer Science, vol. 3, no. 2, pp. 460–478, 2022.

[4] A. F. Siagian and S. Suendri, "Applying the Periodic Review System Method in Progressive Web Apps for E-Commerce Inventory Management," Journal of Information Systems and Informatics, vol. 7, no. 2, pp. 1758–1775, 2025, doi: 10.51519/journalisi.v7i2.1117.

[5] U. S. and P. Kaviya, "Smart Inventory Management System: A Web-Based Solution for Automated Retail Inventory Operations," International Journal of Current Science, vol. 16, no. 1, pp. 444–449, 2026, doi: 10.56975/ijcsp.v16i1.303869.

[6] M. Koren and O. Peretz, "Dynamic Colour Dynamics: Markov Decision Processes for Fashion Inventory Management," Annals of Operations Research, vol. 358, pp. 1329–1359, 2026.

[7] Z. Namazian, J. M. Betts, and P. J. Stuckey, "Predict and Optimize: A Smart Inventory Management Model for Beauty Retail," Optimization Letters, 2025, doi: 10.1007/s11590-025-02226-0.

APPENDICES (IF APPLICABLE)

Appendix A Detailed diagrams: Figure 4.1 defines deployment-layer responsibilities, Figure 5.1 traces standard request creation and Figure 5.2 records principal entity relationships. Sequence/number tables and supplier contacts are supporting schema objects and are omitted from the simplified ER view for readability.

Appendix B Supplementary requirements and schema: FR-01–FR-12 map to the acceptance tests in Table 9.1. The source schema is src/main/resources/db/migration, versions V1–V11. Relevant implementation evidence includes SecurityConfig.java, CustomBagPricingService.java, CustomBagRequestServiceImpl.java, SaleServiceImpl.java and PurchaseServiceImpl.java.

Appendix C Prototype evidence: frontend/src/features/custom-bag/three contains the designer and three model components. App.tsx defines routes and the features folders contain retail screens. README.md describes setup and roles. FINAL_INTEGRATION_REPORT.md, dated 5 October 2026, records the prototype review and its limitations. Figure 7.1 is a wireframe rather than a captured application image.

Appendix D Verification and work plan: target/final-all-tests.log, target/final-default-tests.log and target/surefire-reports contain the existing backend run evidence; target/final-build.log and target/final-lint.log record frontend checks. The four-week remaining plan appears in Section 8.3. Retain database and upload files together when moving the demonstration environment.

FORMATTING AND SUBMISSION GUIDELINES

Use A4 paper; Times New Roman, 12 pt body; 14 pt bold major headings; 1.5 line spacing; justified body text; 1-inch margins; bottom-centre page numbers; numbered sections (1.0, 1.1, 1.1.1); numbered and captioned figures/tables; IEEE references. These specifications are carried over from the supplied 2026–27 synopsis format as a recommended consistent departmental design-report format.

Design emphasis: The report must contain an original overall architecture diagram, clearly defined modules and interactions, appropriate data/database/workflow design, core technical approach, user interface or prototype, technology stack, implementation status and next milestones. Avoid generic diagrams and unsupported implementation claims.